Network Security and Cryptography Botnet Analysis Coursework

Steven Pyper
40319882@live.napier.ac.uk

Abstract. This paper is an analysis of the botnet that was provided for the network security and cryptography coursework. It will include a short review of literature which will be used to form a methodology about analyzing the botnet, both statically and dynamically, and then ways to defend against it in future with IDS rules and firewalling. This report will then show the working network topology which will use a pfSense firewall with 3 networks, one will be considered private and will have one ubuntu machine, one will be considered the DMZ and will contain two machines, a windows 2002 server and a kali machine. With the final network being the default gateway. Then the bot itself with be tested via the methodology created from the research which should show the connection between the botnet controller and agent, communications between them and any other bot behavior which is found. The methodology will contain two types of analysis, dynamic analysis where the bot and controller will be run in a safe manner and using tools like Wireshark capture the network traffic between the devices to find specific commands and responses. Once this is complete static analysis will then be used to verify findings by reverse engineering the bot agent code and finding those commands and responses plus anything else that was not found. Finally, a prototype detection system will be created that will detect botnet activity and stop it without having lots of false positives. This will be completed by using a closed perimeter firewall and IDS rules which will still allow valid traffic but not any from the botnet.

1 Literature Review

1.1 Botnets

A botnet is a collection of computers(called bots) connected to the Internet which have been compromised and are being controlled remotely by an intruder via malicious software. (Saad, 2011), The controller of these bots is often called the botmaster (Abdullah, 2013), Botmasters can often gain full control capabilities on infected machines, giving them the same access to the PC as the legitimate user. (McMillen, 2016), Internet Relay Chat (IRC) has been for a while the most prevalent communication scheme among traditional botnets. (Saad, 2011) however Peer to Peer (P2P) based botnets have been getting more use in the past couple of years, mostly because they can be better secured than IRC channels, with proper encryption. Botnets can be described to have 4 phases “formation, C & C, attack, and post-attack phases.” (Leonard, 2009) As the bot is in our control the formation stage has no importance as the devices become infected by placing the bot on the device, however the C&C and attack phase are more useful, the bots (i.e. infected machines) which are enslaved will receive on a regular basis instructions from the bot master during C&C phase. During the attack phase, the bots will carry malicious activities based on the received instructions. (Saad, 2011) Most methods for detecting botnets happen during the formation or attack phase typically using an IDS (Gu, 2007). There are many ways to detect botnets, there is behavior-based detection systems which focus on how a host reacts to certain data that it receives over the network. (stinson, 2007). This can be further split up into two categories, host based and network-based IDS. These methods however require continuous monitoring the network traffic for indications of bot-infected machines. (Abdullah, 2013). Another way to detect botnets is to set up honeypots, these are devices which are set up to be easy to break into and there should be no normal traffic going to them, that way it is easy enough to detect when a botnet is trying to spread or break into a network. (Provos, 2007). The most common botnet detection method however is passive traffic monitoring, These techniques can be classified as being signature-based, anomaly-based, DNS-based, and mining-based (Feily, 2009). One way detecting botnets is by checking on communications on specific ports however, The problem with port-based identification is the high false identification rates involved and the fact that today there are thousands of network applications that do not use registered TCP/UDP ports (Saad, 2011), this is actually a general problem with most IDS systems that it is difficult to distinguish real traffic from botnet traffic, the false alarm rate is the limiting factor for the performance of an intrusion detection system. (Axelsson, 1999) A good method of analyzing a botnet is through static analysis of the code, specially reverse engineering, particularly if you have access to both the bot and a controller, while this does not work for new bots this report has been created with the fact that both the controller and bot are controller therefor theorically the code can be reversed engineered. In 2014 a paper managed to reverse engineer the citadel botnet which allowed them

to provide additional insights into the functionality, inner workings, and open source components (Rahimian, 2014) by using the findings from the reverse engineering it may be possible to increase the likelihood of detecting the botnet in more detail and lower the false positive rate. For example, the paper describes that by doing static analysis allows you to find embedded images and segments that may be hidden. (Rahimian, 2014). A difficultly explained by this paper is that sometimes the code can be obfuscated which would make it more difficult to get a complete picture of the original code (Rahimian, 2014). One of the most command ways to obfuscate code is to rename all identifiers inside the code(like string names, function names etc), this is called refactoring (Tahiri, 2019) Code obfuscation is standard practice within Malware(and subsequently botnets). It hides the intended meaning of malicious code. (Sinha, 2011). There are many tools that can deobfoscate code so long that the obfuscation method is supposed such as, ida pro and de4dot. These tools will take the obfuscated code and try to make it much simpler to understand by removing the parts that are obfsocated(for example with non-printable characters) and replaces them with sensible alternatives. Code review for security ranks very high on the list of software security best practices (Chess, 2004),while this report focuses on program development this can also apply to botnet analysis, if you are able to find out in detail the code that is run you will be able to get a full understanding of that the program is doing(something that cannot be done through network analysis).

While it is known that the botnet being used for this coursework is not malicious it is still best practice to not allow any botnet traffic to go outside of the network therefor it should be logged and then blocked, except specifically to botnet controller, which should be allowed through the firewall and also logged for analysis. Many botnets will change how they run to try and avoid detection therefor the controller will be run multiple times and analyzed to check if it runs differently each time.

Like most IDS systems, Snort is configured with a set of rules or signatures to log traffic which is deemed suspicious (Feily, 2009). This method has several advantages, such as immediate detection and much lower chance of false positives. However signature-based detection approach is only on well-known Botnets. (Zeidanloo, 2010) and are fairly easy to avoid by making simple changes to the way the bot and controller operate as this will change the signatures. Snort has a advantage with as some rules or signatures being predefined and sometimes you only need to look at the header or the payload not both to create good rules (Haddabi, 2015). Snort performs protocol analysis, content searching and matching which allows it be lightweight and effective while still being able to capture and process many packets (Sagala, 2016).

While is difficult to find many good sources for effective ways to do reverse engineering specifically in the botnet field, as typically with a botnet you do not have access to both the controller and the bot it is only possible to get half of the picture, there are three good papers that have a large sections that cover the reverse engineering of popular botnets that can be used to create a methodology. One of these was the previously mentioned “reverse engineering of the citadel botnet”, in this report they combined static and dynamic analysis that is commonly used when dealing with malware to learn about the structure and functionality as well has behavior monitoring to understand the botnet. (Rahimian, 2014).

The basic methodology they propose consists of three section:

Static analysis:

Disassembling the code and reviewing it to find obfuscated segments, embedded images and decide whether dynamic analysis might be better for code and data decryption. Also, to plan how to deobfoscate and/or decrypt the assembly. Finally want to use Control flow analysis and data flow analysis to understand crypto algorithms and encoding/decoding functionality.

Dynamic Analysis:

Using a debugging environment to execute the binary and set breakpoints, control unpacking and dump process memory which should allow for better analysis. System calls are monitored, network traffic is captured and downloaded files are backed up, the protocol that the bot and controller are communicating on is also observed.

Clone-based analysis process:

Comparing the binary of the bot to other malware and open source bots, this allows for not having to deal with low level assembly(as you have the source code from the opensource bot), this means less time will have to be spent reverse engineering parts of the bot that have already been analyzed. Citadel shared large portions of code to an already analyzed bot called Zeus, they were able to shorten their workload drastically. According to

the first process of the proposed methodology, static and dynamic techniques should be interleaved for advancing the analysis. (Rahimian, 2014) This will also be the first thing that will be completed in this report, getting the code into a readable state.

Another report which followed a similar methodology is "On the analysis of the Zeus botnet crimeware toolkit", they aimed to uncover various obfuscation levels and looking for what the bot would be able to do (such as changes to the operating system, web page traffic, registry etc). One way to look for these changes is to reverse engineer the malware and try to reveal what is hidden behind the assembled code. This analysis provides insight into the inner-working of the crimeware toolkit in general and about the malware binary in particular. (H, 2010) An important note from the report was that if the obfuscation technique used it is easy to remove.

The final report that used reverse engineering for botnets is the "Insights from the analysis of the Mariposa botnet" (Sinha, 2011), their aim was to (1) uncover and remove its obfuscation techniques, (2) circumvent its anti-debugging techniques, (3) understand how the infection and code injection are performed, (4) reveal the functionality of the bot, (5) understand and divulge the inner working of the communication protocol used by the bot and the botmaster. This allowed for a deeper level off understanding of the botnet which allows more targeted defenses to be made, this allows for stopping initial infection of a machine as well as being able to detect already infected machines. These methods in particular allow for knowing not only the communication between the bots but the underlying effect that they have on the machine that they are running on, they can find hidden files and changes that may be made to a machine that would make them more vulnerable.

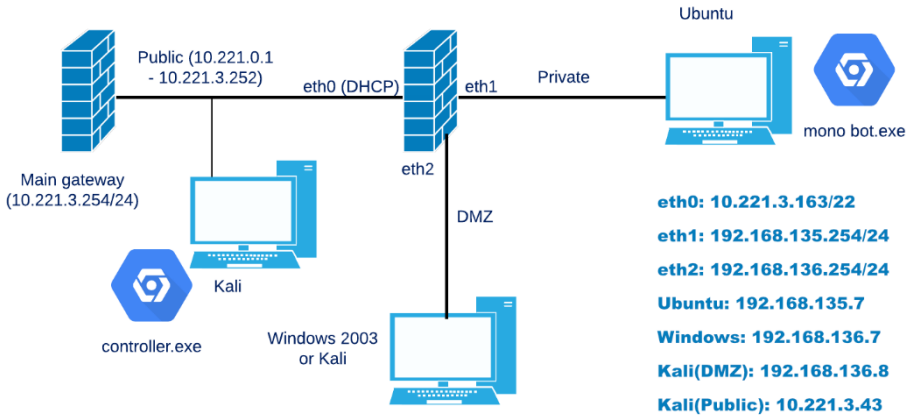
1.2 Methodology

While initially it appears that the report should be focusing on network analysis and then backing up with static analysis based on the handout, the findings made in the literature review show that analysis of the code will be more efficient because there will be a more complete understanding of the bot and controller can be found when compared to analyzing the network traffic as while the network traffic will show the communications between the bot and the controller it will not show the underlying functionality of either, by analyzing the code it is possible to understand exactly what each script is looking for in terms of communication and anything that is only done locally that would not send a response to the other device. (Tahiri, 2019) Therefor the plan should be to get both scripts (bot.exe and controller.exe) and discover as much information about them as possible, this can be done using the file command in Linux, this should give the language they are written in which will allow software to be chosen which supports that language. Depending on the language it may be found that it is easy to reverse engineer (or that it is not). As some languages store there executable code in assembly language they are easier to reverse engineer and as such many also make use of obfuscators which hide variable and method names, c,c++ and c# as well as dotnet, there are deobfuscators that are opensource which could be used depending on the language and if obfuscation method used. If the scripts can be successfully deobfuscated this should essentially give source code (with the only real difference being variable names). (Sinha, 2011) By analyzing the main methods of how the bot and controller operate this should give a solid base understanding of how they both operate and talk to each other. This will allow for more advanced defenses to be made as not only can simple commands be detected that would be seen in the network traffic but also anything that is potentially only seen rarely and could be missed network analysis. While this method will give much higher levels of detail and understanding it will also take much longer to complete than doing network analysis, however network analysis can be used to complement the analysis of the code, the rules that snort will be using are much closer to what will be seen in wireshark. By running the controller and the bot and comparing commands that are seen by using the code the output should be predictable, this can be used to verify the findings from the code as well as to help formulate the snort rules. Depending on the findings the bot or controller may have to be run multiple times, this is another strength of using the code as this will show the inner operation. It may also be possible to create a customized version of the bot or controller that can test individual commands.

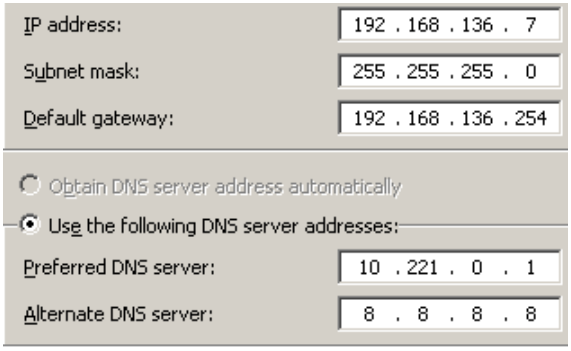
Once an understanding of the commands and anything else are found these can be used to create snort rules which would act as the IDS sensor, these rules should at a minimum detect commands between the bot and the controller and not cause to many false positives but also catch anything that is additional such as data, this specifically should be achievable through the code analysis (Axelsson, 1999). Firewall rules could also be created depending on how the bot operates, if it focuses on only one port or a select few ports these ports could simply be blocked however if it is a popular port or uses many this may not be possible.

2 Network Setup

The following diagram shows the network topology that will be used for the testbed and where the bot and controller will mainly be tested from, although it will also be tested on the windows machine briefly.



Ubuntu:
ifconfig eth11 192.168.135.7 netmask 255.255.255.0
route add default gw 192.168.135.254 eth11
Kali(DMZ):
ifconfig eth0 192.168.136.8 netmask 255.255.255.0
route add default gw 192.168.136.254 eth0
Kali Public:
ifconfig eth0 10.221.3.43 netmask 255.255.252.0
route add default gw 10.221.3.254 eth0
WindowsServer 2003:



pfSense Firewall: assign EM0 to WAN, EM1 to LAN, and EM2 to OPT1
em0
set interface em0, ipv4 dhcp
interface em1 192.168.135.254/24
interface em2 192.168.136.254/24

After all the devices had been configured the pfSense firewall would have to be modified to allow connections from the private space to the public space. This was done by using the ubuntu machine and connecting into the pfSense machine to edit rules using the GUI, to allow these connections NAT would have to be implemented using a 1:1 mapping.

NAT 1:1 Mappings

	Interface	External IP	Internal IP	Destination IP	Description	Actions
✔	WAN	10.221.3.163	*	*		

Once this was completed the devices were then tested to make sure they could all access the internet and that there was no issues with the network, all devices could connect to google therefor the network was operating, the main test was making sure that ubuntu could ping the public kali machine as that is where the controller will be running which it was able to do. For security reasons as mentioned in the research firewall rules will be implemented to block any traffic coming in or going out expect to the public kali machine, this will stop any unwanted traffic coming into the network and also any “malicious” packets from leaving the controlled network. By using the following firewall rules only connections between the pfSense machine and public kali are allowed. By disabling the first rule no connections are allowed at all and kali cannot contact the pfsense machine, when the first rule is then enabled the contact is allowed.

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue
✔	0/2 KiB	IPv4 *	10.221.3.43	*	*	*	*	none
✖	0/0 B	IPv4 *	*	*	*	*	*	none

The rules for LAN and OPT1 below

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway
✔	0/3.47 MiB	*	*	*	LAN Address	80	*
✔	0/0 B	IPv4 *	10.211.3.43	*	*	*	*
✔	0/4 KiB	IPv4 *	*	*	10.221.3.43	*	*
✔	0/85 KiB	IPv4 *	LAN net	*	OPT1 net	*	*
✔	0/0 B	IPv6 *	LAN net	*	OPT1 net	*	*
✖	0/133 KiB	IPv4 *	*	*	*	*	*

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway
✔	0/0 B	IPv4 *	10.221.3.43	*	*	*	*
✔	0/0 B	IPv4 *	*	*	10.221.3.43	*	*
✔	0/464 B	IPv4 *	OPT1 net	*	LAN net	*	*
✔	0/0 B	IPv6 *	OPT1 net	*	LAN net	*	*
✖	0/124 KiB	IPv4 *	*	*	*	*	*

With these rules only connections between internal devices and connections to public kali are allowed and everything else will be blocked. This has been tested and access to the internet has been cut on all devices but they can still access the kali machine, this allows for a safe testbed to be used.

3 Botnet Analysis

3.1 Static Analysis

Understanding the code

As both the controller and the bot are available this allows for detailed analysis of the code assuming it is in a format that is readable. The code is the best way of understanding what the bot is attempting to do as it cannot do anything outside of the code. To analyse the code the first thing that needs to be found is the language that it is written in and any meta data about the file. The simplest way to do this is to use the file command in the kali machine. By doing this is clear that both files are Mono/.Net assembly, this means that programs are written in .net and programs such as JetBrains DotPeek (a .net decompiler) should be able to decompile the code very quickly. (https://www.jetbrains.com/decompiler/)

Decompiling.

By decompiling both programs it is clear that an obfuscation method has been used to make it harder to understand what is doing on in both the bot and the controller. When copying code there are non-printable characters that are being used as variable names which is a telltale sign that the code has been obfuscated. While there is enough detail to get a basic understanding of some of the lines like

```
System.IO.File.Delete(\u0005.\u0002(1461507325));
```

Showing that some sort of file gets deleted, it does not provide a clear picture of what file is deleted and in more complicated functions it would become confusing very quickly. To work around this the code will have to be deobfuscated, to do this an opensource program called de4dot can be used(https://github.com/0xd4d/de4dot), this will allow for discovering the obfuscation method used and then deobfuscating the programs so long as they are supported.

Deobfuscation

Both the programs were tested using de4dot to detect the obfuscation method method, and both returned stating that they were obfuscated using Eazfuscator.NET 3.3.149 which is a method supported by de4dot. After deobfuscation it appears that the controller has been deobfuscated successfully however it initially appears that the bot still had not been as it had issues as things like the string arrays still did not contain strings just numbers, with this in mind it would be good to look for telltale signs of another obfuscator which would be easier in an editor so both programs were exported into JetBrains Rider (https://www.jetbrains.com/rider/ a .net code editor), suddenly it appeared to fix itself and update itself in both programs to be fully readable, a potential reason for this happening that when it gets ready to export the code it does not want to store the numeric fake value and the true value separate and at that point it updates and places the true value where it should be. This does mean that the code that is now available is essentially the source code and as such it is easy to analyse. This allows for a complete understanding of the connection between the bot and controller as well as any extras that may not be discovered by dynamic analysis.

Controller findings

The main class would create a new class which would create a listener listening to any ip address coming in on port 5002, it starts a loop that waits for a connection.

String Arrays/Strings.

The controller contains three global strings which contain large quantities of text, it contains upper and lower case letters, numbers from 0 to 9,slashes and pluses, this indicates that it may be a base64 encoded string, as they both share a similar start to the string, by converting R0lGODlhA from base64 to normal text it results in the text GIF89a, this indicates that these two strings of text are in fact GIFs. By using an online conversion tool

(https://codebeautify.org/base64-to-image-converter) these strings can be converted into images, the first string is shown in Fig. 1. , and the second string in Fig. 2.

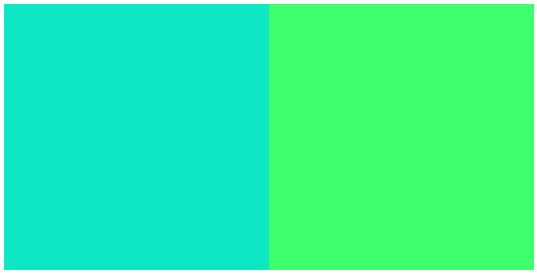


Fig. 1. **Fig. 2.**

The third global string follows a similar structure to the previous images however instead starts with /9j/, by converting the string in the same process as before it was discovered that this image started with FFD8FF which is the magic number for a jpg file meaning this is also an image shown in Fig. 3.

Fig. 3.

There are four more string arrays that are defined when the connection has started, three of these are in plain text, the first string array contains

"the", "owls", "are", "not", "what", "they", seem"

the second string array contains

"animal", "Hobbes", "is", "KEY", "what", "Calvin", "pet"

The third string array contains

"apple.com", "microsoft.com", "ibm.com", "twitter.com", "hpe.com", "bbc.co.uk"

The fourth string array contains three strings which are very long and appear to be ciphered. Initial analysis made it seem like the strings were ciphered using a ceaser method, however after trying all possible keys no readable string was created. However further analysis showed there was a substring that said “SXDXRXQFXES SPRIPA-ZJ-STGGMCT (SOSS)”, the SOSS is likely an abbreviation of the previous characters, as can be seen the 4 words start with sszs, this does not match the abbreviation and as such is likely a method where they key changes each character like a vigenere cipher. A online vigenere decoder (www.dcode.fr/vigenere-cipher) was used as it has the ability to use the kasiski method which is used for predicting key sizes and the key itself, it is

able to using the fact that the key will repeated to the side of the plaintext. If they key is 3 characters long it is essentially 3 Caesar ciphers that are joined together, this means that frequency analysis still works, by finding any repeating substrings it is possible to find the factor of the key which will give the possible lengths of the key. By using this method all three strings have been successfully as shown appendix(viginere cipher strings):

There are two more string arrays towards the end of the controller, the first contains 7 messages

"The distributed denial of service attacks against dynamic domain name service provider Dyn this morning have now resurged. The attacks have caused outages at services across the Internet."

"The botnet, made up of devices like home Wi-Fi routers and Internet protocol video cameras, is sending massive numbers of requests to Dyn's DNS service. Those requests look legitimate, so it's difficult for Dyn's systems to screen them out from normal domain name lookup requests."

"An advanced piece of malware, known as Regin, has been used in systematic spying campaigns against a range of international targets since at least 2008. A back door-type Trojan, Regin is a complex piece of malware whose structure displays a degree of technical competence rarely seen. Customizable with an extensive range of capabilities depending on the target, it provides its controllers with a powerful framework for mass surveillance and has been used in spying operations against government organizations, infrastructure operators, businesses, researchers, and private individuals."

"Hundreds of thousands of Android phones have been infected with malware that uses handsets to send spam and buy event tickets in bulk. Mobile security firm Lookout said the virus, called NotCompatible, was the most sophisticated it had seen. The cyberthieves behind it had recently rewritten its core code to make it harder to defeat, it said. Mobile malware aimed at smartphones is steadily getting more complex, said security company Wandera. Jeremy Linden, a security analyst at Lookout, said: \"The group behind NotCompatible are operating on a different plane to the typical mobile malware maker.\""

"Prince added that Cloudflare was seeing a sizable increase in errors in traffic for its customers because the attack was affecting infrastructure providers like GitHub. \"If a customer's site is pointing to a git there, now we can't reach Github,\" he said. \"There are definitely infrastructure providers that we can't reach."

"At least twenty people have died in a wave of protests against social and economic inequality which began more a than a week ago.";

"Speaking on Sunday, President Donald Trump said Abu Bakr alBaghdadi detonated his suicide vest after fleeing into a tunnel, chased by US military dogs.";

The last string array contains 17 city names from the uk

"edinburgh", "glasglow", "stirling", aberdeen", "inverness", "london", leeds", birmingham", "bristol", "bath", carlise", "derby", manchester", york", "belfast", "bangor","brighton"

Methods and functionality

Method_0: accepts the connection from the listener on port 5002, it then prints out the number of the connection this is(increments by one for every person that connects, the ip address and port of the bot and then the ip address of the controller.

Method_1(intro): this is the main method that controls how the controller operates and will be explained at the end of this section once all the other methods it will call have been explained.

method_2: although it is not called in current version it is implemented and only a small change would need to make it run; by recreating the function it will create a request to a website stored in string_4, the response stream will be stored, it will then be converted into base64, if this response is smaller than 10000 bytes the base64 string will be padded with all A's, this could be used as a way to hide updates to the bot as this is done on the controller, this could be streamed to the bot and used as a way to modify or as a way to get around fire-

walling rules, if the firewall was blocking specific ports or protocols the controller could create the request and hide it as a base64 transfer in a connection that is allowed through. The base64 string is returned as as a string (not as a stringbuilder).

Method_3: takes in a string that is converted into a character array and characters are swapped based on two random numbers created, that character array is converted into a string which is then changed to be all upper case and returned.

Method_4: if the string_4 is null is it is returned as an empty string. If it is not null, it will look at string_4 and take a character out of that string based on the index, that character is then used to test string_5 to find the first occurrence of that character and take the index that occurrence, finally that index is used to add a character that is that index far in inside string_6 to a string, if string_6 is too short for that index the original character is added. This is looped for the length of string_4, This string is then converted to upper case and returned.

Method_5: takes in a streamwriter and a string, takes that string and flushes it, essentially takes the input and sends it to the tcp connected device.

Method_6: another method that isn't called but if it was would put the thread to sleep for a time between two int's in seconds.

Method_7: Checks if a file called out.dat exists, if it does it changes it to a normal file and deletes it. No matter if it existed before or not it will then create a new file and write string_3(the /9j/ image) to the file called out.dat and then make the file hidden.

Smethod_0: takes in a string and a int, converts the string to lower, it is a Caesar cipher method, if the key is less than 0 26 is added to it, it takes each character and shifts it by the key, uses the ascii numbers and puts the shifted character into a string and returns that string.

Smethod_1: contains the string array of 7 messages, picks one based on the random function(smethod_6) and returns it.

Smethod_2: contains the 17 cities in a string array, returns one based on the random function(smethod_6), returns it in lowercase.

Smethod_3: computes a md5 hash for a utf8 string, and stores the hash in bytes, they are then converted into hex and returned as a hex string.

Smethod_4: computes a SHA1 hash for a utf8 string, and stores the hash in bytes, they are then converted into hex and returned as a hex string.

Smethod_5: computes a SHA256 hash for a utf8 string, and stores the hash in bytes, they are then converted into hex and returned as a hex string.

Smethod_6: locks object0 and returns a random number between two numbers, if flag is successfully and the number is returned, it unlocks object0

How the controller Functions

Method_1: contains the two plaintext strings arrays and the vigenere cipher string array. When the connection is made a streamwriter and reader is created with the tcp stream from the connection using ascii encoding. It converts whatever is received into lowercase so that it can be tested.

If what is received starts with "hello" it responds with "Welcome to the gang…" to the bot.

If what is received starts with "get" it responds with the jpg image in base64 format

If it starts with "test" it creates a random number between 0 and 6, this is then modulus'd by 3, if the remainder is 0 it calls the sha-1 hash method, if the remainder is 1 it calls the sha256 hash method, if the remainder is 2 it calls the md5 method, all three of these hash methods use the strArray1(one with the owls are not what they seem) as the word to hash. The hash is then sent to the bot.

If it starts with failover it sends "that took a while" to the bot.

If it starts with connect it gets a random number between 0 and 5, gets a string from strArray3 (apple.com, Microsoft.com etc) based on the random number as the index. It then sends that string to the bot.

If it starts with takedown it generates a random number between 0 and 2, it gets a string from the string array of the vignere ciphers with the random number as the index, it then sends that string to the bot.

If it starts with capture it creates a string of characters from a-z, it then shuffles those characters and saves them in a new string_6, it creates another string called string_4 which is created by using method_4 with a random string from smethod_1, the a-z characters and string_6. This should return a string which is just a scrambled alphabet, however it actually causes an error, it is able to create the randomized alphabet but fails when

trying to do the for loop in method_4, by testing the character a, it creates a index2 of 0, however when it tries to add to the previously empty string that is when it fails so there must be an issue with the line “empty += (string) (object) string_6[index2];”, when testing with the original bot the command seems to work and it does return a string successfully so this is likely broken from the deobfescation.(this was later fixed by removing the (string)) When the bot is running it works as expected shuffling the ordering of the strings stored in smethod_1.

If it starts with keepalive it will create a new string based of the string array containing the cities in smethod_2, and a random number between 0,24. The cities will then be cesar ciphered with the key being the random number. This will then be sent to the bot.

If it starts with loop send “All Explained Successfully ... IV all the zeros" to the bot

If it starts with look send both the found base64 gifs separated with a space to the bot.

If it starts with code send "Calvin never forgets transmogrification animal ... may be key..." to the bot.

If it starts with gener send “bgure pbzznaqf znl or ninvynoyr ... uvag uvag” to the bot. the repeating characters at the end are likely repeating words which indicates a ceaser cipher or another cipher with another cipher with a repeating key. By trying all of the keys for a ceaser cipher it was found that with a key of 13 it gave the “ouput other commands may be available ... hint hint”

If it starts with snoop send "mmmmm...my army grows" to the bot.

If it starts with hint, create a md5 hash based on the string array with animal hobbes is key… using the random function.

If it starts with good, writes sleep to the bot and ends the connection.

If it starts with v0 it calls gclass0, this is another ciphering method where it takes in one of the strings from the string array in smethod_1, one of the city names from smethod_2 and a int value of one(which acts as a multiplier so can be pretty much ignored), essentially uses the city name as the key to vignere the cipher.

If none of the previous is true it will return an empty string to the bot

If there is an exceptions it increments a int value and if that int reaches 11 flag1 is set to true which will force the controller to break out of this connection. This seems to be the way it ends communications, the error in testing that comes up is “System.NullReferenceException: Object reference not set to an instance of an object.”

Bot findings

Only runs if there is an argument added when script is started, uses two ints, creates a connection to the ip address that is taken from the argument on port 5002(which is static). Then calls the main controlling section of the program.

Strings

String_0 contains:

"HELLO", "V0", "TEST", "FAILOVER", "CONNECT", "TAKEDOWN", "CAPTURE", "KEEPALIVE", "LOOK", “CODE", “GENERATE", "GET", "SNOOP", "LOOP", "GOODBYE", "QUIT"

Method_0 strArray contains:
"3bbf158fca09f2e3ab0cf47ea53d2505582cef56c807a10816b9f10d7f9384c61da757baa5391580974abd673d8323c7b416ed3e6d45d29b4902ef60f095b9ac9cc06c01ffc28eb4c02f5236e123f185a65c032c4897c7af1643658ac0f652e7",
"ad68a660b4325f130b32d9ef64e5d5d61fd4f61ad1b20f999b3c0c2b77f761c3509e9828b4f2ca6256655fba468da36da8c57309a1ff9931cf27d08fd832c82c2f1a50030ab608985201570bb2036619"

these are likely RSA as there are 3072 bits in the first string and 2560 with the second. This could mean that these are public and private keys? Possible key pair? Attempted to use these as private keys and public keys as well as tried to do some AES decryption based on the hints that are found throughout the code however no possible key could be find, the strings could not be decrypted which leads to the assumption that possibly there is actually a issue with the way that the strings have been encrypted

Method_7 str contains: a jpg image in base64 format (too long to put in report, found in images folder)

Methods and functionality

Method_0: main class deciding everything.

Method_1:never called, would read a file or connection to the controller and read it all, would then print the first 30 characters of what is received, if there is any issue with that it will print saying recier timed out and the error message.

Method_2: takes in a string and an int, returns the string up to the length of that int or the length of the string whatever is smaller.

Method_3: puts the thread to sleep for a second, sends a string to the controller and prints a "." to the console

Method_4: takes in an int array, randomizes and shuffles the order of it.

Method_5: never called, would use the dns table to look for ip addresses and look for the internetwork family and return that address.

Method_6: never called, would create a ping to an ip address, and return if it was successful or not.

Method_7: ouputs the image to output.dat and makes it hidden

Method_8: takes in two strings, one contains content and the other contains a filename, checks if the file exists, if it does delete it. Then no matter if it existed or not, create that file and put the content in it and close the file, then turn it into being a hidden file.

Method_9: puts thread to sleep for a random time between two numbers in seconds.

Smethod_0: creates a temporary file in the temporary file folder and returns that filename.

Smethod_1: locks object_0 and returns a random number between the two ints passed in.

Method_0: creates a int array based string_0, index 0 = 0, 1=1 all the way up to 15, it then uses the shuffling function to shuffle the ordering of the int array so it is not ordered from 0 to 15. It then creates a timeout time of 5 seconds. It then starts looping 15 times (for each of the int array). It then gets the string that is stored in the value of that int array (so for the first time whatever value is stored int_0[0], it then uses that value to get the string from str1).

If the command is quit or bool_1 is changed to being true further in the program it will close the connection and end the program.

If the command is failover it locks object_0 and generates a number between 5000 and 4000000, creates a double array that is as long as the random number that is generated. Then uses a for loop to put the number inside that double array that is the same as its index (1=1 etc). It then calls method_7 which outputs the image into out.dat, it then adds to the command string adding a space and the number that was generated as a 32bit int.

If the command is loop it picks one of the (possible)RSA key pair strings randomly and creates a tmp file. There is another if loop however this will never run as it is in an else if setup, if this was to run it would try to create a connection to 10.211.3.250 on port 53 which would be the dns.

If the command is generate, this adds the name of the machine, the current directory, the OS version and the user that is logged on.

If it gets to this point and the command is not snoop, it checks if the command is look, if it is it creates a txt file called look.txt and places the command string into that file. If it starts with sleep, it prints +] to the console. And closes the connection

if it gets to this point and has not closed the connection it sends the command string to the controller.

If the command from the controller does not start with goodbye it writes ~ to the console, if it does start with goodbye it closes the connection.

Summary of controller findings

Contained 3 images, two of these were simply hex coloured images and the other was a image of a phonebox, By doing an image lookup for this image it is a banksy artwork that was removed, the people in the image are supposed to be 1950's spies snooping on the person in the phonebox. This image likely refers to the fact that the controller is able to snoop in on the devices in the network. This also hints at there being information that is not obvious to the person reading this likely meaning there is information hidden either in the images or the text so all images will have their metadata checked for hidden information.

The string array's the owls are not what they seem are from a from a show called Twin Peaks, in this they have received a string which appears to be in base64 and have the message the owls are not what they seem, the other section of this that happened afterwards is cooper, this likely could be the key to some method of decryption, it could be that cooper is the code that was used to generate the RSA key.

The third string contained links to common websites like Apple and Microsoft , these links are sent to the bot, the bot however does not do anything with these links but if the bot was modified slightly could be used as a way to DDoS one of these websites by using many infected machines and sending the link to attack that website.

The large vigenere ciphered messages would back this up with the first talking about botnets and there connection to distributed denial of service, the others talked about lockscreen ransomware and a ransomware trojan both of which theorically could effect this network, if the botnet is able to make a connection to the controller there is no real reason the controller could not craft a packet which would update the bot and add in this ransomware.

Three smethods were found that would hash the same string, that being the owls are not what the seem, this means that these can be hashed using all three methods(MD5,SHA-1 and sha256) and can be tested for in the snort rules.

MD5 is also used on the second string (animal hobbes etc) so these can also be hashed and tested.

There are many unused functions in the controller which indicates that the bot has been modified over time or that the intent is to make it harder to understand the program, particularly when it was obfuscated.

There also appears to be many hints throughout with strings like "all explained successfully IV all the zeros" which refers to Initialization vectors likely meaning that somewhere in the controller or bot there is a reference to AES key/s, this could mean that the two keys that were found before that were thought to be public and private keys could infact be a AES key and a RSA key. This would mean that "Calvin never forgets transmogrification animal ... may be key..." could also be a hint, referring to Calvin and Hobbes the comic strip having something to do with the key, could be something like tiger(hobbes) or something that the transmogrification could turn into like an elephant.

The ceasar ciphered message that deciphered to "other commands may be available… hint hint" is likely for network-based analysis as there is no way to hide the commands in the code.

Finally the most important thing is that the port number is hardcoded to be 5002 for the listener, that means that by simply blocking the port, connection between the controller and bot can be severed, however this is potentially only a short-term fix as if the attacker can get back inside the network they may be able to update the bot to use a new port number.

Summary of bot findings

Contained a string that contains commands: "HELLO", "V0", "TEST", "FAILOVER", "CONNECT", "TAKEDOWN", "CAPTURE", "KEEPALIVE", "LOOK", "CODE", "GENERATE", "GET", "SNOOP", "LOOP", "GOODBYE", "QUIT"

These commands are used in both the bot itself and in communications with the controller.

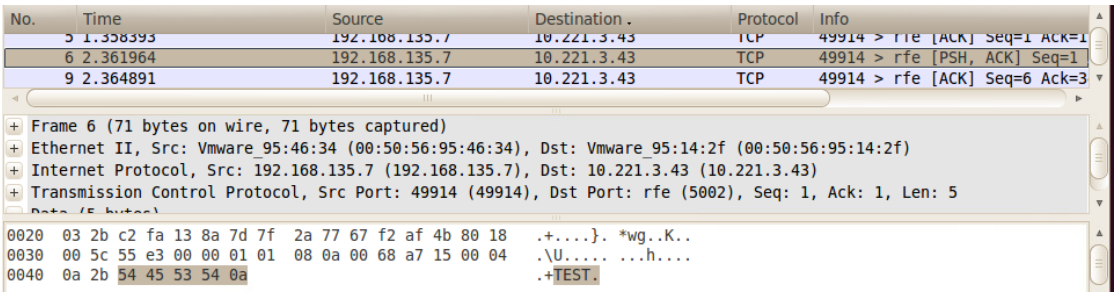
The bot contained a string array containing two strings which are likely to do with some sort of encryption method, the best predication is either AES based on the hints found in the controller, or that they are in fact public and private RSA keys, theorically it could also be a RSA key and encrypted string.

There was also another base64 image which gets stored in a out.dat, when the metadata was analyzed there was a comment stating "create your own bot to access extra cmds ...run cmds many" which is similar to the other hint in the controller("other commands may be available… hint hint") again trying to hint that there are some commands that may not be shown in network analysis.

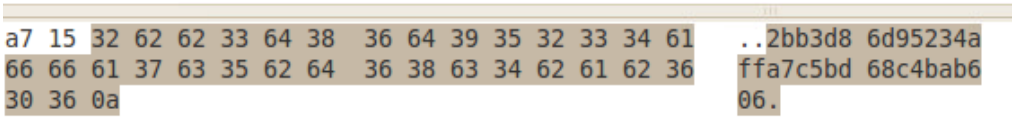
One of the biggest security issues from both the controller and the bot is the generate command in the bot, this would send the machine name, current directory, operating system and its version plus the username of the person logged in to the controller, this information would allow the person in control of the controller to find which exploits exist on the current version of the operating system, it would also allow them to be able to find what directory they are giving more information to the controller and what permissions might be expected for that user(if they are a simple employee which folders would the have access to etc). It should also be noted that the controller did not look for the command generate but for the command starting with gener , while this does not affect the running of the bot or controller it should be noted that tests for generate should also follow tests for just gener as this is the more dangerous command even though they will not happen with the current version.

3.2 Dynamic Analysis

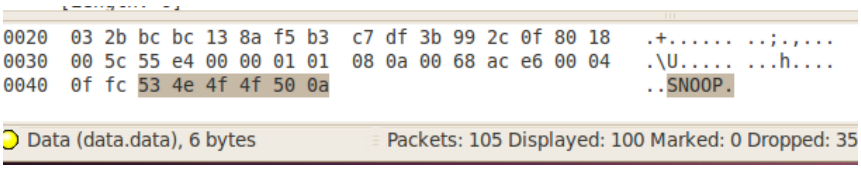
As we now know what to expect from the bot and the controller this can be tested by running them and analysising network traffic using Wireshark and using commands that are sent to predict the response or the opposite where there is a response and try to figure out the command that was sent. The bot will be run 3 times so that there are a mix of commands and data to be tested on.



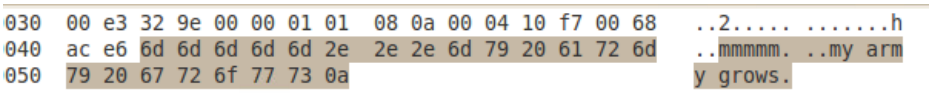
As the command test was used the expected output would be a md5/sha1/sha256 hash and as can be seen by the next image this is what happened



Which is a md5 hash for “they” which is one of the hashes that would be expected.
Another example is the following, the command snoop was found



Therefor the expected message in responds should be “my army grows”



Which is exactly what was sent.

Doing this method through Wireshark is very time consuming so a better way is to telnet to the controller and send individual commands, that way the requests and responses are not random. That way specific commands that have single output responses can be tested to make sure everything from the code analysis can be valided.

The list of commands that will be tested are:

“hello”, “failover”, “loop”, “code”, “gener”, “snoop” and finally “good” which will close the connection.

The expected responses from the deobfuscated code to all these commands (in the same order) are:

”Welcome to the gang…”, “that took a while…”, “All Explained Successfully ... IV all the zeros”, "Calvin never forgets transmogrification animal ... may be key...", “bgure pbzznaqf znl or ninvynoyr ... uvag uvag" "mmmmm...my army grows", “sleep…”

The following is the output from the test:

```
hello
Welcome to the gang...
failover
That took a while...
loop
All Explained Successfully ... IV all the zeros
code
Calvin never forgets transmogrification animal ... may be key...
gener
bgure pbzznaqf znl or ninvynoyr ... uvag uvag
snoop
mmmmm...my army grows
good
sleep...
Connection closed by foreign host.
root@ubuntu:~#
```

This shows that the network traffic is easy to predict by using the code meaning that there is now very little that can go unnoticed. As all the commands return exactly as predicted that means that the code itself can be used for generating snort rules as well as verifying that the listener port for the controller is only ever on port 5002 which means that a firewall rule to stop that port will function well.

The other way to verify findings from the bot are to use the look and failover commands, these should generate two files on the ubuntu machine out.dat and look.txt which as the following shows are generated and out.dat contains the image that it is supposed to.

```
root@ubuntu:~# ls -l out.dat
-rw-r--r-- 1 root root 60289 2019-12-20 07:29 out.dat
root@ubuntu:~# ls -l look.txt
-rw-r--r-- 1 root root 5 2019-12-20 07:29 look.txt
root@ubuntu:~#
```

It was also found that the program did not appear to run different depending on the device it was running on, this is due to it being written in .net which is cross platform, this means that any device could be the controller or bot which means defenses will have to be made to protect all devices on the network.

4 Prototype Defenses

4.1 Implementation

There are two main ways that defenses could be implemented based on the findings that were found. The simpler way would be to simply lockdown port 5002, by researching what applications use this port number there are 3 main applications that use this port, Yahoo voice chat and yahoo messenger voice chat which both use UDP and radio free ethernet which uses tcp and udp, this would mean that port 5002 could be blocked for tcp traffic only and would not have too many false positives. However this is not a long term solution as this only cuts the connection between the bot and the controller, if the intruder is able to update the port number(which as seen by the code would only require changing a few numbers) they would be able to have control again therefor while this method does work, it should also be noted that as discussed in the literature review there are many

unregistered applications that could use this port and as such it has a higher false positive rate (Saad, 2011) .IDS rules should also be used to look for suspicious commands and expected outputs. This will allow for any connection on any port number to be check and if it is matched either logged or dropped, this allows for seeing that there is an issue on a machine and what machine it is rather than just stopping the connection. The IDS rules should also track some of the content passed as well as the commands that are used. (Gu, 2007)

4.2 IDS Rules/Signatures.

(all the IDS rules can be found in the appendix)

There are many commands which get sent between the all of which are easily detectable by using Snort, by writing simple rules it should be possible to find the commands used and for some commands even the information passed (such as the words that are hashed in multiple ways). As the firewall will be blocking communication on port 5002 these rules will not be limited to a port number, the snort rules are a backup for if the controller and bots port number is changed. These rules could be made more general by using all ips and ports being any rather than using the ip address of the ubuntu network, however for testing this can be used as there is only one device on that network, if this was to be used on other networks the $home_net should be replaced with the ip address of the host wanting to be tested, or replaced with any, $external_net could also be used on the second any for the destination ip however will not affect testing. The rules will detect any commands that are sent from bot to the controller can be seen in the appendix. With these rules all outgoing connections from the bot to the controller are caught. There will also have to be rules that will detect commands/data coming from the controller, these commands can easily be found by using the analysis done previously or by modifying the deobfuscated controller and printing the output to the console as shown below.

```
That took a while...
Welcome to the gang...
apple.com
```

The rules that detect commands sent from the controller to the bot can be found in the appendix (IDS RULES). As there are some words which will be hashed using different methods the hashed version of each word will have to have a rule for each hashing method as well, the first array("the","owls","are",not",what",they","seem" will need be hashed in three different methods which means there will be 21 rules just for finding these hashes. A script will be created to make all these hashes and this output will be used to form the rules (see appendix). The rules that are created by the script contain those from the hashing methods as well as those from the ceasar cipher, however after completing the cesar cipher this increased the rule count to around 500 as it could find each word and key possible. therefor the rules are split up into two main categories, the commands from the bot and controller and the expected outputs. The reason there are two categories is that the bot and controller have clearly been put through some changes as time has went on and as such it would not be surprising if the command names changed but the content stayed the same, or the opposite with the content changing but the commands staying, by having both of these sets the snort rules will have more longevity. Due to the number of rules they will not all be put in the appendix although enough will be shown for each section to show the output from the scripts and the ones that were completed manually and while it would be possible to continue automating the snort scripts for the vignere cipher there are already many rules which should track nearly everything adding this would add very little additional accuracy.

4.3 Firewalls

As it was discovered the only port number that was used for this specific botnet is tcp port 5002 which is used for few other programs therefor simply blocking the port number both internally and externally will mean that even if a machine inside the network (like on the dmz) becomes infected there is no chance that it can communicate with other devices on the network or with a device outside of it. On the WAN network(public) port 5002 will be blocked as a destination and as a source as this will mean if the controller ends up inside the network and devices outside the network try connecting to it this will not be possible. It will be blocked on opt

and lan as this will stop internal connections such as if the controller is in the dmz and the bot is on ubuntu and tries to connect to it internally. As there is a 5 second timeout for the bot the packets will be blocked rather than rejected, this will mean that there is not a response to the bot saying that the network is rejecting the packet and the program will stop after those 5 seconds.

The following rules will be implemented that will completely lock down port 5002 on all networks (these rules are on wan, lan and opt1), these rules will all have the log button enabled so that if there are any connections on this port they should be logged and easy to discover.

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue
	0/0 B	IPv4 TCP	*	*	*	5002	*	none
	0/0 B	IPv4 TCP	*	5002	*	*	*	none

Once these were implemented the bot (and the telnet method) were run again to make sure no connection could be made and that the firewall was working as intended, this was attempted from multiple all of the devices inside the network(ubuntu, windows and kali) and none of them could make a connection to the external kali device, this was also attempted with the controller running internally but the firewall rules stopped the connection.

The firewall restrictions that were put in place for testing at the beginning of this report had to be lifted so the block all source and destination from all three networks were disabled, along with the rules allowing access to 10.221.3.43 as they are no longer needed, This will allow the machines to access the internet again.

Finally, some normal firewall rules should be so that devices on the dmz cannot contact those in the private but those in the private network can still access the webserver on dmz. On the OPT1 section of the firewall they allow any rules will be disabled and instead replaced with a block rule for any packet with the destination of lan, this will stop any new connections being made from dmz to private however connections that are made from private to dmz will be given a response(as they are established connections)

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queu
	0/0 B	IPv4 *	*	*	LAN net	*	*	none

Additionally as the windows server is running a webserver there should be a method that will create a static ip address for the windows server, the best way to do this would be to create a port forward rule where any connection that comes to the pfSense machine on port 80 is forwarded on to the windows machine, this should allow the kali public machine to connect to the webserver

	Interface	Protocol	Source Address	Source Ports	Dest. Address	Dest. Ports	NAT IP	NAT Ports	D
	WAN	TCP	*	*	WAN address	80 (HTTP)	192.168.136.7	80 (HTTP)	

The final firewall commands can be found in the appendix.

5 Testing

SNORT

The firewall new firewall rules were disabled for testing.
To test the snort rules snort was enabled by using:

snort -c 2.rules -I eth11 -l log/ -k none

this allowed it to track incoming and outgoing communications which would mean the rules worked for bot and controller commands. To find the accuracy of the snort rules Wireshark was left open to run and track the traffic, snort was run as shown above and then the communication between the bot and the controller was left to run. It was successfully able to capture all commands between the bot and controller and was able to capture KEEPALIVE, HINT, TAKEDOWN and TEST responses(like the hashes) and identify them, in the log was a keepalive command sent by the bot, followed by the keepalive response being the word brighton with a key off one

```
[**] [1:10000003:0] KEEPALIVE COMMAND [**]
[Priority: 0]
12/19-12:10:51.719955 192.168.135.7:40995 -> 10.221.3.43:5002
TCP TTL:64 TOS:0x0 ID:186 IpLen:20 DgmLen:62
***AP*** Seq: 0x84EEB6EB  Ack: 0xB076BA21  Win: 0x2C8  TcpLen: 32
TCP Options (3) => NOP NOP TS: 6161760 6553844

[**] [1:10000143:0] CONTROLLER KEEPALIVE RESPONSE Word: brighton. Key:1 [**]
[Priority: 0]
12/19-12:10:51.720735 10.221.3.43:5002 -> 192.168.135.7:40995
TCP TTL:63 TOS:0x0 ID:19447 IpLen:20 DgmLen:61
***AP*** Seq: 0xB076BA21  Ack: 0x84EEB6F5  Win: 0xE3  TcpLen: 32
TCP Options (3) => NOP NOP TS: 6554095 6161760
```

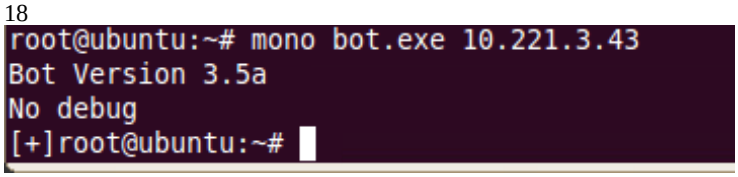
By running the bot for large periods of time and testing the output of snort when compared to the wireshark traces it was continuously able to detect the bot (or telnet commands), it was obviously not able to detect responses that were not put in snort rules like the viginere ciphers and scrambled alphabet however it was able to find the command that was used to get that response. There was a very low false positive rate for most of the rules, however this was one rule which caused lots of false positives, this was the “GET” command which is used, in hindsight this is very obvious as this is used in webpage requests which means that it will generate a false positive for every webpage request. To fix this it would be best to change the GET Rule specifically to not look at HTTP/HTTPS ports, this would negate the false positives that were found entirely.

Firewalls

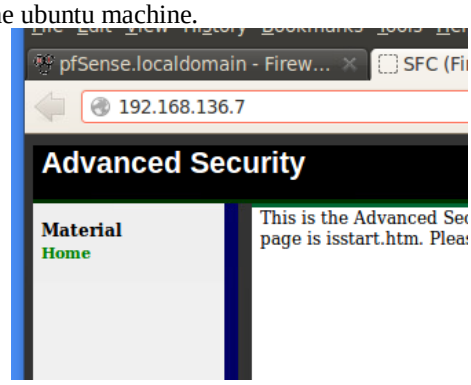
Once the firewall rules were implemented the bot (and the telnet method) were run again to make sure no connection could be made and that the firewall was working as intended, this was attempted from multiple all of the devices inside the network(ubuntu, windows and kali) and none of them could make a connection to the external kali device, this was also attempted with the controller running internally but the firewall rules stopped the connection.

```
root@ubuntu:~# telnet 10.221.3.43 5002
Trying 10.221.3.43...
telnet: Unable to connect to remote host: Connection timed out
root@ubuntu:~# telnet 10.221.3.43
Trying 10.221.3.43...
telnet: Unable to connect to remote host: Connection refused
root@ubuntu:~#
```

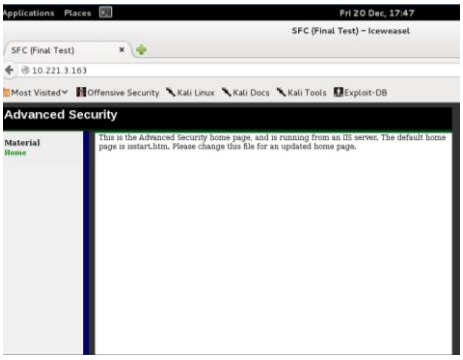
This shows that port 5002 was blocked and therefor the connection timed out with no response but if you just tried to telnet the machine itself it would refuse which is a response showing the machine is there and is able to be connected too.



All the devices were then tested to make sure they could access the internet and all devices were able to access google showing that the firewall rules were not blocking ordinary traffic. Ubuntu was also able to access the webserver that was running on windows however windows was not able to ping or initiate a connection with the ubuntu machine.



The final test of the firewall/nat was seeing if the kali public was able to connect to the windows server which it was able to do as shown below.



All the firewall rulings that were implemented have been implemented successfully and work as intended. Connection between the bot and the controller is now stopped in its current configuration. Dmz devices cannot connect to private devices, however private can connect to dmz, all devices can access the internet, the windows webserver is able to serve pages to those that can connect to the pfSense machine, the communication line between the bot and the controller has been severed by blocking port 5002 and the snort rules would detect any attempt from the bot to connect to the controller even if it was on a different port number and detect any commands that would be sent.

6 Conclusion

From the literature review it was found that there are many ways for a botnet to operate such as peer to peer or through IRC, while these are all slightly different the general idea of breaking into a network and making gains is the same. Therefor protection is needed to specifically against bots during the C&C and attack phase. During the C&C and attack phases the bots carry out malicious activtieis baed on instructions that are sent from the controller. IDS is a great way to detect and log these malicious activities in a network as it can use either host based or networked based detection. Host based detection requires continuous monitoring of the network traffic. The most common botnet detection method however is passive traffic monitoring, these techniques can be classified as being signature-based, anomaly-based, DNS-based, and mining-based. The problem with port-based identification is the high false identification rates involved and the fact that to-day there are thousands of network applications that do not use registered TCP/UDP ports. Another method of analyzing a botnet is through static analysis of the code, especially reverse engineering, this has been done before with many different botnets like zeus, Citadel and Mariposa, these methods all focus on getting the code from being in assembly into a more readable format by unpacking and deobfoscating the code, it is also possible to compare parts of the bot that are copied from opensource versions which can lower the amount of work required to analyse the botnet. This report took a similar stance to those reports because the controller and bot were both available to analyse increasing the opportunity to gain all the information between the bot and controller, which would be worth the required time to deobfoscate and understand the code rather than focusing on network analysis and simply locking the bot out of the network. Firstly the language the program was found to be dotnet, this narrowed down the possible obfuscates that could be used and a opensource program was able to detect that it was obfuscated using Eazfuscator.NET 3.3.149 which was a supported method of de4dot, this allowed both the controller and bot to be returned to near source code, a dotnet editor was used which allowed for the programs to be displayed in a easier to read format which allowed many finds from both controller and bot. The controller contained 3 images, two gifs (which were sent together with a space between them) and a jpg image which was a banksy artwork of 1950's spies snooping on a person in a phonebox. There were also 3 long strings which contained already Viginere ciphered text (found in appendix). There were many more strings inside the controller which were used for hashing or ciphering when being sent to the bot. the controller was able to hash strings using three methods, MD5, SHA1 and SHA256, it could also make use of Caesar ciphers and viginere ciphers. Sometimes the controller will just send strings to the bot as commands for it to act on depending on the input it receives from the bot, there are many different strings and commands that were found that were sent to the bot with many different hashes, images or strings being sent.

The bot contained an image of toys in base64 format that when put through a metadata reader contained a comment stating "create your own bot to access extra cmds …run cmds many". There was also a command which would create a file and place an image inside that file, there again were also many commands and strings that were send to the controller. The biggest security issue that was found inside this bot was the generate command as this would sent the machine name, current directory, operating system(and its version) plus the username of the person logged in to the controller, this would allow the controller to find vulnerabilities inside that system which could be abused by the fact there is already an open connection there allowing them to take full control of the system.

Both the controller and bot had many unused methods which indicates that the bot has been updated and changed over time and/or that the unused methods are there as a way to make it harder for people to understand the program particularly if they have not been able to successfully deobfuscated the code, this could be seen as the reverse of the honeypots discussed in the literature review where these are here to catch people out and keep their attention by making them spent more time analyzing these methods. They also would only communicate on port 5002 as it was hardcoded into the bot and controller which means that blocking the port number was something to be considered.

To check that the findings from the reverse engineering was accurate and had not been negatively effected by the de/obfuscation process network analysis was also completed, by connecting the bot and controller together and monitoring network traffic it should be possible to predict the output or the command by checking any packet at random, this was completed and Wireshark analysis showed that the reverse engineering of the bot had been successful and was indeed accurate. With the accuracy of the findings it was then possible to use these to

create accurate snort and firewall rules that would not cause too many false positives. By implementing snort rules which looked for the commands (and didn't care about ip or port numbers) it was possible to capture every communication between the bot and the controller as well as some of the more advanced snort rules were able to capture exactly what was transmitted (such as the hashed words or the ciphered text). While the general false positive rate was very low there was one snort rule which created many false positives, this was the GET command, this was a oversight when creating the rule as it checks all tcp traffic that contains get, this means that it would find every web request as the get command, the simple fix to this was to make sure it was not looking at any http/s ports specifically for that rule, this would make the snort rules much better in the long term. As the bot only used one port number to communicate with the controller creating a firewall rule that would block port 5002 is an effective way to cut communication between the bot and the controller, as communication is cut there should theorically be no way for the controller to update the bot to use a different port number. As this port is used by few applications there should also few false positives that are created by this method. Initially the only interface that would need to block port 5002 was the WAN interface however this would mean that if the controller was inside the network it would still be able to communicate with bots that were also inside that network, therefor the blocking of port 5002 was also included on LAN and OP1. The packets will be blocked not rejected, that way the bot will not know if there is a controller outside the network or not, this should lead to the bot simply running for 5 seconds, getting no reply and then switching off, this means that the snort rules should still pick up the attempted connection from the bot so its still possible to find there has been an infected machine inside the network. By testing the implementation of the firewall rules the connection between the bot and controller was indeed cut and the bot would run, get no response and then close. The additional firewalls that were implemented to allow connection from private to dmz but not from dmz to private also functioned correctly and the public kali device was able to access the web server that was on the windows machine due to the port forwarding rule implemented on the pfSense machine. Further testing of the snort rules proved that they were very effective at discovering commands that were sent between the controller and bot missing out on no commands although there was some data that was missed as they were not implemented into the rules.

7 Appendix

7.1 Viginere Cipher string

String 1

FAPDLPDXYX HPH NSNUXCQES ISET HDXI OU ISI ICUCESIGFGTJGP VEHEZRSXQWI FDG ELE SXDXRXQFXES SPRIPA-ZJ-STGGMCT (SOSS) PIEECZH LKAXCDX DNC ORS LTCI BDIYITH RZQPGDXMSTS MC MXGLM MPAHERT. BTVAX QZXNTID AEGT AVEKXZYSAN FWES XY HDDH LXTPRVW AVPTRSI HPGUGXEC RTHPERRWPV BGXLR KGTMW' BADR ORTQD SN HTNYRXIJ ENS UCINRW TRTTGYIT HTCZIRT LRD WDDXICV AVOKXOIR DKS. QIGPT QAALLVE IPCKEIH TRTTGYIT DU ELICVD (MOI) SPZIRTD PIZT CSUITCW, DXVTXAA KTHED GPGOGSD (HVGH), LRD LTMGABH/DICJGTXY RPXIRPH, PRSAPGMNV KLWT CJXFEGH ZJ TWTDI DTKTGEH XYXO P QZXNTI, HLIRW TW TWTY YSTS ES CDCOYCI SOSS PIEECZH. QPAHWASICI SES RDYJIGBPH TWPE ET ATLWT HDXI OU ISI DTKTGEH JDID XC ELE SNY HNH PEXARZD ERT SGVS, UJCXHTG XETRWTRG IWP XERWYMCPA TRDXRLXOGH LRD IPNXIRH, EICWCTUUTH, LRD EGZGESJCIS PHDSCXPEID LXEL PGTGMOJH VROLC XMRPX MSTCTE ETIPNOS.

Has the key apple and becomes:

flashpoint has confirmed that some of the infrastructure responsible for the distributed denial-of-service (ddos) attacks against dyn dns were botnets compromised by mirai malware. mirai botnets were previously used in ddos attacks against security researcher brian krebs' blog krebs on security and french internet service and hosting provider ovh. mirai malware targets internet of things (iot) devices like routers, digital video records (dvrs), and webcams/security cameras, enslaving vast numbers of these devices into a botnet, which is then used to conduct ddos attacks. flashpoint has confirmed that at least some of the devices used in the dyn dns attacks are dvrs, further matching the technical indicators and tactics, techniques, and procedures associated with previous known mirai botnet attacks.

String 2

HYEEK EFV TJU XMGEF UJ FRNFUQKRRR – RSQBSPXISE RNTWCDWNXI OED RTGFPPGOSB IAAYSANAEK. PCTKFIVSVN EGRGFMJGVS JHBCW O WUYR-WQIERT QSJSNMI HYAG VVSMEAZW MFU SXSA RCPKWGZNT ESII PP UV TZLRY. MH JALY CCL HNBI HF PNE QCEEL (G VOESBS) XC XEG GGQVSF ZS MFUE VG OXAVT. IBTRLVXWFN EGRGFMJGVS THNTKSJ YBAV TZLRY WC POH IEB'K OCKR HYEZ. OX RFEF ZLWJ BL KRQIYCZMBX TUK JWCEF – YIS KHR JIHRIYY JCI EAZIFGRVYIG JEPZMCE IS ESI'IE VTXSIEFZIR ZN GNI HVCUTSZFGVKW OED GKGVEIDAIG NE'IK WSVN. BRHSI VRXWWFNF UJ FRNFUQ IJUNRPM TLNOQ MFU UGZS UOAK WCDEGNMBX IYRIURL JOXV POHX TQ, RNQ ZLOK YBA EFV BRORU WIAKH PP A CUPWTE SUVQV OE MSJVRASIBK ATKRQP. TUKWS TLNOQG RRR LEZJE. VZ MG R SPGVS KAPZMQ UEFOKBVD GU QOBE LUY DRY GNI AFNRE AWKHBAX HVLYORU RNLURS NHB SMUYT OK EPCE GU VSJTBXI MFUE VG. BVWRX ZSISVURG VNPXCDK TUK JWCEF UR MFUE VG GF YBA GOE'T NIGSJS GNIA, RNQ ZLSE SVSTZP DRSEBU MBTIM KO EKWHFRR ESII FVRIG.

Has the key orange and becomes:

there are two types of ransomware – lockscreen ransomware and encryption ransomware. lockscreen ransomware shows a full-screen message that prevents you from accessing your pc or files. it says you have to pay money (a ransom) to get access to your pc again. encryption ransomware changes your files so you can't open them. it does this by encrypting the files – see the details for enterprises section if you're interested in the tech-nologies and techniques we've seen. older versions of ransom usually claim you have done something illegal with your pc, and that you are being fined by a police force or government agency. these claims are false. it is a scare tactic designed to make you pay the money without telling anyone who might be able to restore your pc. newer versions encrypt the files on your pc so you can't access them, and then simply demand money to re-store your files.

22

Third string:

RVYRADPOERTV IU H GENUVBAATL IVOLHC XHCA IERILIW CQTEYTGYH VUPUXRG OPRVOUVUX WKUSSWU, ITPIGCTH TQ OPZE HPGWT DLTR PQZIID VV ILE KUIIRPLI SN 5 ULEXEOITV 2013. CTFEXONVROET WGSPCNPXEF CXE IPMTGTGK TQAKS PXTCJWQEPAH, ENF CXE AP LMMSVPCK BQACIT; YOTR AEAXZAVLS, XHG TPPWCYT INEYNTTU JTVTCPC XYRLH SF HPAIS UADVEF VC POEHA ENF TDYNVLS REVDDVK FYXZEU BHMNI YHE PWIAMC-MLN GRAWISGTHELY, YPIL TJL EVIXHII KGF HXOTLS SNNF DR TJL BELYHGI'S EVCXRQS HIRXLGW. TJL BELYHGI TJLC HIUWAEYU H BISUHVI WJPRL OHMTVS VV SICTFEX TJL SETC PU E PCFBINV (AWVOWNW IIVOTV BKARSIP VG E PTL-EEIF JPWH XVJGHGY) XW MCKT FY C ZIETGK SIAFSXRE, CUS XHTLPXEPLS XO FLAITG AWI PTPKETG RTC IH AWI DGHSPIPL EESULH. MF VOT HECKAMNG PH ROV TTX, TJL BELYHGI OHMTVEF AD HEEYNTT FHIE VKH PR OPSXRE ULGZIEL EVOXPSID DF ILE OHAAATL'H SPGYPXOTZ, USR C ZXKNKMXGAPAAC HKNWIR RYXGE KU QMTEVXR.

Has key peach and becomes:

cryptolocker is a ransomware trojan that targets computers running microsoft windows, believed to have first been posted to the internet on 5 september 2013. cryptolocker propagated via infected email attachments, and via an existing botnet; when activated, the malware encrypts certain types of files stored on local and mounted network drives using rsa public-key cryptography, with the private key stored only on the malware's control servers. the malware then displays a message which offers to decrypt the data if a payment (through either bitcoin or a pre-paid cash voucher) is made by a stated deadline, and threatened to delete the private key if the deadline passes. if the deadline is not met, the malware offered to decrypt data via an online service provided by the malware's operators, for a significantly higher price in bitcoin.

7.2 Scripts/Snort Rule Generators

These scripts were used to create snort rules that would have been time consuming to create by hand, this allows for more specific rules that are looking for specific communications between bot and controller

CONNECT SNORT RULE GENERATOR (used as a base method for testing if this idea would work)

```
string[] strArray3 = new string[3]
    {
      "FAPDLPDXYX HPH NSNUXCQES ISET HDXI OU ISI ICUCESIGFGTJGP",
    "HYEEK EFV TJU XMGEF UJ FRNFUQKRRR – RSQBSPXISE RNTWCDWNXI ",
    "RVYRADPOERTV IU H GENUVBAATL IVOLHC XHCA IERILIW CQTEYTGYH "

    };
    int sid = 10000110;
    for (int index = 0; index < strArray3.Length; index++)
    {
      Console.WriteLine("alert tcp any any -> $HOME_NET any
(msg:\"CONTROLLER TAKEDOWN RESPONSE" +
                        "\";content:\"" + strArray3[index] +
"\";sid:" + sid + ";)");
    }
```

TEST SNORT RULE GENERATOR

```
string[] strArray1 = new string[7]
 {
   "the",
   "owls",
   "are",
   "not",
   "what",
   "they",
   "seem"
 };

 int commandcount = 0;
 int sid = 10000070;
 MD5 md5_0 = MD5.Create();
 SHA1 sha1_0 = SHA1.Create();
 SHA256 sha256_0 = SHA256.Create();
 Console.WriteLine("#md5 hashes start here");
 for (int index = 0; index < strArray1.Length; ++index)
 {
   //md5 hashing
   byte[] hash =
md5_0.ComputeHash(Encoding.UTF8.GetBytes(strArray1[index]));
```

```
   StringBuilder stringBuilder = new StringBuilder();
   for (int hashindex = 0; hashindex < hash.Length; ++hashindex)
   {
     stringBuilder.Append(hash[hashindex].ToString("x2"));
   }
   Console.WriteLine("alert tcp any any -> $HOME_NET any
(msg:\"CONTROLLER TEST RESPONSE WORD(md5)  " + strArray1[index] +
                     "\";content:\"" + stringBuilder.ToString() +
"\";sid:" + sid+";)");
   sid++;
   commandcount++;

 }
 Console.WriteLine("#md5 hashes end here");

 Console.WriteLine("#sha1 hashes start here");
 for (int index = 0; index < strArray1.Length; ++index)
 {
   //sha1 hashing

   byte[] hash =
sha1_0.ComputeHash(Encoding.UTF8.GetBytes(strArray1[index]));
   StringBuilder stringBuilder = new StringBuilder();
   for (int hashindex = 0; hashindex < hash.Length; ++hashindex)
   {
     stringBuilder.Append(hash[hashindex].ToString("x2"));
   }
   Console.WriteLine("alert tcp any any -> $HOME_NET any
(msg:\"CONTROLLER TEST RESPONSE WORD " + strArray1[index] +
                     "\";content:\"" + stringBuilder.ToString() +
"\";sid:" + sid+";)");
   sid++;
   commandcount++;

 }
 Console.WriteLine("#sha1 hashes end here");
 Console.WriteLine("#sha256 hashes start here");
 for (int index = 0; index < strArray1.Length; ++index)
 {
   //sha256 hashing

   byte[] hash =
sha256_0.ComputeHash(Encoding.UTF8.GetBytes(strArray1[index]));
   StringBuilder stringBuilder = new StringBuilder();
   for (int hashindex = 0; hashindex < hash.Length; ++hashindex)
```

```
    {
      stringBuilder.Append(hash[hashindex].ToString("x2"));
    }
    Console.WriteLine("alert tcp any any -> $HOME_NET any
(msg:\"CONTROLLER TEST RESPONSE WORD " + strArray1[index] +
                      "\";content:\"" + stringBuilder.ToString() +
"\";sid:" + sid+";)");
    sid++;
    commandcount++;
  }
  Console.WriteLine("#sha256 hashes end here");
```

HINT SNORT RULE GENERATOR

```
string[] strArray1 = new string[7]
{
  "animal",
  "Hobbes",
  "is",
  "KEY",
  "what",
  "Calvin",
  "pet"
};
 int commandcount = 0;
 int sid = 10000070;
 MD5 md5_0 = MD5.Create();
 Console.WriteLine("#md5 hashes start here");
 for (int index = 0; index < strArray1.Length; ++index)
 {
   //md5 hashing
   byte[] hash =
md5_0.ComputeHash(Encoding.UTF8.GetBytes(strArray1[index]));
   StringBuilder stringBuilder = new StringBuilder();
   for (int hashindex = 0; hashindex < hash.Length; ++hashindex)
   {
     stringBuilder.Append(hash[hashindex].ToString("x2"));
   }
   Console.WriteLine("alert tcp any any -> $HOME_NET any
(msg:\"CONTROLLER HINT RESPONSE WORD(md5)  " + strArray1[index] +
                      "\";content:\"" + stringBuilder.ToString() +
"\";sid:" + sid+";)");
   sid++;
   commandcount++;

 }
```

```
string[] strArray = new string[17]
{
  "edinburgh",
  "glasglow",
  "stirling",
  "aberdeen",
  "inverness",
  "london",
  "leeds",
  "birmingham",
  "bristol",
  "bath",
  "carlise",
  "derby",
  "manchester",
  "york",
  "belfast",
  "bangor",
  "brighton"
};
int sid = 10000110;

for (int index = 0; index < 25; index++)
{
  for(int index2 = 0; index2 < strArray.Length;index2++)
  {
    string ciphered = Class1.smethod_0(strArray[index2].ToLower(),
index);
    Console.WriteLine("alert tcp any any -> $HOME_NET any
(msg:\"CONTROLLER KEEPALIVE RESPONSE Word: "
        + strArray[index2] + ". Key:" + index + "\";content:\"" +
ciphered + "\";sid:" + sid + ";)");
    sid++;
  }

}
```

7.3 Final Snort Rules

Bot and controller commands

```
var HOME_NET 192.168.135.0/24
var EXTERNAL_NET !$HOME_NET

#outgoing bot commands
alert tcp $HOME_NET any -> any any (msg:"LOOK COMMAND";content:"LOOK";sid:10000001;)
alert tcp $HOME_NET any -> any any (msg:"CONNECT COMMAND";content:"CONNECT";sid:10000002;)
alert tcp $HOME_NET any -> any any (msg:"KEEPALIVE COMMAND";content:"KEEPALIVE";sid:10000003;)
alert tcp $HOME_NET any -> any any (msg:"FAILOVER COMMAND";content:"FAILOVER";sid:10000004;)
alert tcp $HOME_NET any -> any any (msg:"CODE COMMAND";content:"CODE";sid:10000005;)
alert tcp $HOME_NET any -> any any (msg:"TAKEDOWN COMMAND";content:"TAKEDOWN";sid:10000006;)
alert tcp $HOME_NET any -> any any (msg:"LOOP COMMAND";content:"LOOP";sid:10000007;)
alert tcp $HOME_NET any -> any any (msg:"GENERATE COMMAND";content:"GENERATE";sid:10000008;)
alert tcp $HOME_NET any -> any any (msg:"TEST COMMAND";content:"TEST";sid:10000009;)
alert tcp $HOME_NET any -> any any (msg:"GOODBYE COMMAND";content:"GOODBYE";sid:10000010;)
alert tcp $HOME_NET any -> any any (msg:"V0 COMMAND";content:"V0";sid:10000011;)
alert tcp $HOME_NET any -> any any (msg:"HELLO COMMAND";content:"HELLO";sid:10000012;)
alert tcp $HOME_NET any -> any any (msg:"SNOOP COMMAND";content:"SNOOP";sid:10000013;)
alert tcp $HOME_NET any -> any any (msg:"CAPTURE COMMAND";content:"CAPTURE";sid:10000014;)
alert tcp $HOME_NET any -> any any (msg:"GET COMMAND";content:"GET";sid:10000015;)

#CONTROLLER COMMANDS
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HELLO RESPONSE";content:"Welcome to the gang";sid:10000020;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER GET RESPONSE";content:"/9j/4AAQSkZJRgABAQEASABIAAD/2wBDAAICAgICAgICAgICAgICAwMDAgIDAwQDAwMDAwQFBAQEBAQEBQUGBgcGBgU";sid:10000021;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER FAILOVER RESPONSE";content:"That took a while";sid:10000022;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER LOOP RESPONSE";content:"All Explained Successfully";sid:10000023;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER LOOK RESPONSE (gif 1)";content:"R0lGODlhAAQABP8AAA3nxHR973dumpNsGwUP4MNjVc4lLa";sid:10000024;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER LOOK RESPONSE (gif 2)";content:"R0lGODlhAAQABP8AAD//bxJ8B1v7cAJDT1Qwkx9rjaUmTEkJces";sid:10000025;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER CODE RESPONSE";content:"Calvin never forgets transmogrification animal";sid:10000026;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER GENER RESPONSE";content:"bgure pbzznaqf znl or ninvynoyr ... uvag uvag";sid:10000027;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER SNOOP RESPONSE";content:"my army grows";sid:10000028;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER GOOD RESPONSE";content:"sleep...";sid:10000029;)
```

TEST command reponses

```
#md5 hashes start here
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD the MD5";content:"8fc42c6ddf9966db3b09e84365034357";sid:10000040;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD owls MD5";content:"7b4601ac7f6fe54f86669548acc46286";sid:10000041;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD are MD5";content:"4015e9ce43edfb0668ddaa973ebc7e87";sid:10000042;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD not MD5";content:"d529e941509eb9e9b9cfaeae1fe7ca23";sid:10000043;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD what MD5";content:"4a2028eceac5e1f4d252ea13c71ececc6";sid:10000044;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD they MD5";content:"2bb3d86d95234affa7c5bd68c4bab606";sid:10000045;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD seem MD5";content:"85814ce7d88361ec8eb8e07294043bc3";sid:10000046;)
#md5 hashes end here

#sha1 hashes start here
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD the SHA1";content:"bbccdf2efb33b52e6c9d0a14dd70b2d415fbea6e";sid:10000047;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD owls SHA1";content:"6227bd0181a4790fc5426b744cf088f0571beaed";sid:10000048;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD are SHA1";content:"5f9e5802bbabc71f9275fccf122f5a52b8599f12";sid:10000049;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD not SHA1";content:"557f255516719ea16f8f4a0aae1166054e2c9b43";sid:10000050;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD what SHA1";content:"a110e6b9a361653a042e3f5dfbac4c6105693789";sid:10000051;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD they SHA1";content:"f1fdd46d8d229558fa889e1610ec4ca9adc25328";sid:10000052;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD seem SHA1";content:"72f15b250fdd58ccfae958dace1213f71d6d41ad";sid:10000053;)
#sha1 hashes end here

#sha256 hashes start here
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD the SHA256";content:"b9776d7ddf459c9ad5b0e1d6ac61e27befb5e99fd62446677600d7cacef544d0";sid:10000054;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD owls SHA256";content:"beda14763de0969a064921bfd97d425e109d47e21dd6085bd86f2e89f91cfae6";sid:10000055;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD are SHA256";content:"ba78973ddcf98d4e5369f5e722d681d94f5106895e5d6cf6fa3ca8240fabdc14";sid:10000056;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD not SHA256";content:"254bb97b57f12e1608fefc4517de768427b2fd6d2cffbbfcbc09f3c818198d5f";sid:10000057;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD what SHA256";content:"749ab2c0d06c42ae3b841b79e79875f02b3a042e43c92378cd28bd444c04d284";sid:10000058;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD they SHA256";content:"151e58a831306903b282edb477bb6c4b713887cdd2b0353c84aff6f43da8e95d";sid:10000059;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TEST RESPONSE WORD seem SHA256";content:"d26b980930289c5944632503be7bf9c1745435b31a4e0a1de1b7ca7a2913a0d5";sid:10000060;)
#sha256 hashes end here
```

HINT command reponses

```
#MD5 hashes start here
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  animal";content:"1e4483e833025ac10e6184e75cb2d19d";sid:10000070;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  Hobbes";content:"48a47a95e16adbba58888df370ff0324";sid:10000071;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  is";content:"a2a551a6458a8de22446cc76d639a9e9";sid:10000072;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  KEY";content:"3b5949e0c26b87767a4752a276de9570";sid:10000073;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  what";content:"4a2028eceac5e1f4d252ea13c71ececc6";sid:10000074;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  Calvin";content:"ae7a9f31f296ffd4935056cfa158fa3a";sid:10000075;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER HINT RESPONSE WORD(md5)  pet";content:"6c43c0a88fbf0f44ba944d00524e45c3";sid:10000076;)
#md5 hashes end here
```

KEEPALIVE CEASAR SNORT RULES

(note it should say controller keepalive response not takedown, but there are approximately 430 rules with that error and it should be obvious with the word and key that it is acesar cipher, this realistically should be changed in the code and then reapplied but copying with the virtual machines leads to a formatting issue which takes a long time to go through each line and fix)

these commands go from key 1 all the way to key 24 which is the highest possible from the code and cover ever single word.

```
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bangor. Key:1";content:"CBOHPS";sid:10000142;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: brighton. Key:1";content:"CSJHIUPO";sid:10000143;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: edinburgh. Key:2";content:"GFKPDWTIJ";sid:10000144;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: glasglow. Key:2";content:"INCUINQY";sid:10000145;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: stirling. Key:2";content:"UVKTNKPI";sid:10000146;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: aberdeen. Key:2";content:"CDGTFGGP";sid:10000147;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: inverness. Key:2";content:"KPXGTPGUU";sid:10000148;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: london. Key:2";content:"NQPFQP";sid:10000149;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: leeds. Key:2";content:"NGGFU";sid:10000150;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: birmingham. Key:2";content:"DKTOKPIJCO";sid:10000151;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bristol. Key:2";content:"DTKUVQN";sid:10000152;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bath. Key:2";content:"DCVJ";sid:10000153;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: carlise. Key:2";content:"ECTNKUG";sid:10000154;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: derby. Key:2";content:"FGTDA";sid:10000155;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: manchester. Key:2";content:"OCPEJGUVGT";sid:10000156;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: york. Key:2";content:"AQTM";sid:10000157;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: belfast. Key:2";content:"DGNHCUV";sid:10000158;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bangor. Key:2";content:"DCPIQT";sid:10000159;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: brighton. Key:2";content:"DTKIJVQP";sid:10000160;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: edinburgh. Key:3";content:"HGLQEXUJK";sid:10000161;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: glasglow. Key:3";content:"JODVJORZ";sid:10000162;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: stirling. Key:3";content:"VWLUOLQJ";sid:10000163;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: aberdeen. Key:3";content:"DEHUGHHQ";sid:10000164;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: inverness. Key:3";content:"LQYHUQHVV";sid:10000165;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: london. Key:3";content:"ORQGRQ";sid:10000166;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: leeds. Key:3";content:"OHHGV";sid:10000167;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: birmingham. Key:3";content:"ELUPLQJKDP";sid:10000168;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bristol. Key:3";content:"EULVWRO";sid:10000169;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bath. Key:3";content:"EDWK";sid:10000170;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: carlise. Key:3";content:"FDUOLVH";sid:10000171;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: derby. Key:3";content:"GHUEB";sid:10000172;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: manchester. Key:3";content:"PDQFKHVWHU";sid:10000173;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: york. Key:3";content:"BRUN";sid:10000174;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: belfast. Key:3";content:"EHOIDVW";sid:10000175;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bangor. Key:3";content:"EDQJRU";sid:10000176;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: brighton. Key:3";content:"EULJKWRQ";sid:10000177;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: edinburgh. Key:4";content:"IHMRFYVKL";sid:10000178;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: glasglow. Key:4";content:"KPEWKPSA";sid:10000179;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: stirling. Key:4";content:"WXMVPMRK";sid:10000180;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: aberdeen. Key:4";content:"EFIVHIIR";sid:10000181;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: inverness. Key:4";content:"MRZIVRIWW";sid:10000182;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: london. Key:4";content:"PSRHSR";sid:10000183;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: leeds. Key:4";content:"PIIHW";sid:10000184;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: birmingham. Key:4";content:"FMVQMRKLEQ";sid:10000185;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bristol. Key:4";content:"FVMWXSP";sid:10000186;)
alert tcp any any -> $HOME_NET any (msg:"CONTROLLER TAKEDOWN RESPONSE Word: bath. Key:4";content:"FEXL";sid:10000187;)
```

Firewall Rules

WAN

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description
✖	0/0 B	IPv4 TCP	*	*	*	5002	*	none		
✖	0/0 B	IPv4 TCP	*	5002	*	*	*	none		
✔	0/29 KiB	IPv4 *	*	*	*	*	*	none		
✔	0/0 B	IPv4 TCP	*	*	192.168.136.7	80 (HTTP)	*	none		NAT

LAN

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue
✔	1/628 KiB	*	*	*	LAN Address	80	*	*
✖	0/0 B	IPv4 TCP	*	5002	*	*	*	none
✖	0/600 B	IPv4 TCP	*	*	*	5002	*	none
✔	0/5 KiB	IPv4 *	LAN net	*	*	*	*	none

OP1

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue
✖	0/0 B	IPv4 *	*	*	LAN net	*	*	none
✖	0/0 B	IPv4 TCP	*	5002	*	*	*	none
✖	0/0 B	IPv4 TCP	*	*	*	5002	*	none
✔	0/0 B	IPv4 TCP	*	*	! LAN net	*	*	none

References

Abdullah, R. S., 2013. *Preliminary study of host and network-based analysis on P2P Botnet detection,* Bandung, Indonesia: IEEE.

Axelsson, S., 1999. *The base-rate fallacy and its implications for the difficulty of intrusion detection,* s.l.: CCS '99 Proceedings of the 6th ACM conference on Computer and communications security.

Chess, B., 2004. *Static analysis for security,* s.l.: IEEE.

Feily, M., 2009. *A Survey of Botnet and Botnet Detection,* Athens: IEEE.

Gu, G., 2007. *BotHunter: Detecting Malware Infection Through IDS-Driven Dialog Correlation,* s.l.: Georgia I.

Haddabi, F., 2015. *On the Effectiveness of Different Botnet,* s.l.: Springer international publishing switzerland.

H, B., 2010. *On the analysis of the Zeus botnet crimeware toolkit,* Ottawa: IEEE.

Leonard, J., 2009. *A Framework for Understanding Botnets,* Fukuoka: IEEE.

McMillen, D., 2016. *The inside story on botnets,* s.l.: IBM X-Force® Research.

Provos, N., 2007. *Virtual Honeypots: From Botnet Tracking to Intrusion Detection,* s.l.: Pearson Education.

Rahimian, A., 2014. *On the Reverse Engineering of the Citadel Botnet,* s.l.: LNCS.

Saad, S., 2011. *Detecting P2P botnets through network behavior analysis and machine learning,* Montreal: IEEE.

Sagala, A., 2016. *Automatic SNORT IDS rule generation based on honeypot log,* s.l.: Chiang Mai.

Sinha, P., 2011. *Insights from the analysis of the Mariposa botnet,* s.l.: IEEE.

Sinha, P., 2011. *Insights from the analysis of the Mariposa botnet,* s.l.: IEEE.

stinson, e., 2007. *Characterizing Bots' Remote Control Behavior,* s.l.: LNCS.

Tahiri, S., 2019. *Reverse Engineering Obfuscated Assemblies [Updated 2019].* [Online] Available at: https://resources.infosecinstitute.com/reverse-engineering-obfuscated-assemblies/ [Accessed 20 decemeber 2019].

Zeidanloo, H., 2010. *A taxonomy of Botnet detection technique,* s.l.: IEEE.